

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»**
(БГТУ им. В.Г. Шухова)

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем



Лабораторная работа №2
по дисциплине: Теория информации
тема: Исследование кодов Шеннона-Фано

Выполнил: ст. группы ПВ-221

Дорошенко Владимир Юрьевич

Проверили: Твердохлеб В. В.

Белгород 2024

**Задание №1 Построить код для сообщения, содержащего строку
панграммы «в чащах юга жил бы цитрус? Да, но фальшивый экземпляр!».**

Символы	Вероятности	Этапы					
	0.16	0	0	0			
а	0.09			1			
л	0.07		1	0	0		
и	0.05				1		
р	0.036			1	0	0	
ы	0.036					1	
в	0.036				1	0	
Д	0.018					1	
б	0.018	1	0	0	0	0	
г	0.018					1	0
е	0.018						1
ж	0.018				1	0	
з	0.018					1	0
й	0.018						1
к	0.018			1	0	0	
м	0.018					1	0
н	0.018						1
о	0.018				1	0	0
п	0.018						1
с	0.018					1	0
т	0.018						1
у	0.018		1	0	0	0	
ф	0.018					1	0
х	0.018						1
ц	0.018				1	0	0
ч	0.018						1
ш	0.018					1	0
щ	0.018						1
ь	0.018			1	0	0	
э	0.018					1	0
ю	0.018						1
я	0.018				1	0	0
!	0.018						1
,	0.018					1	0
?	0.018						1

Коды для каждого символа

Символы	Коды
	000
а	100
л	0010
и	1010
р	00110
ы	10110
в	01110
д	11110
б	00001
г	010001
е	110001
ж	01001
з	011001
й	111001
к	00101
м	010101
н	110101
о	001101
п	101101
с	011101
т	111101
у	00011
ф	010011
х	110011
ц	001011
ч	101011
ш	011011
щ	111011
ь	00111
э	010111
ю	110111
я	001111
!	1001111
,	011111
?	111111

Задание №2 Построить код для сообщения, содержащего строку «Victoria nulla est, Quam quae confessos animo quoque subjugat hostes»

Символы	Вероятности	Этапы											
	0.132353	0	0	0									
s	0.102941			1									
u	0.102941		1	1									
a	0.0882353			0	0								
o	0.0882353				1								
e	0.0735294	1	0	0	0								
t	0.0588235			1									
n	0.0441176			1	0								
q	0.0441176				1	0							
i	0.0441176				1								
c	0.0294118		1	0	0	0							
l	0.0294118				1								
m	0.0294118				1	0							
,	0.0147059				1								
Q	0.0147059			1	0	0	0	0					
V	0.0147059					1			1				
b	0.0147059					1	0						
f	0.0147059						1			1			
g	0.0147059				1	0	0	0					
h	0.0147059						1			1			
j	0.0147059					1	0						
r	0.0147059						1			1			

Коды для символов

Символы	Коды
	000
s	100
u	110
a	0010
o	1010
e	0001
t	1001
n	0101
q	01101
i	11101
c	00011
l	10011
m	01011
,	11011
Q	000111
V	100111
b	010111
f	110111
g	001111
h	101111
j	011111
r	111111

Программа для нахождения кодов символов, дисперсии и коэффициента сжатия:

```
#include <iostream>
#include <string>
#include <locale>
#include <vector>
#include <unordered_map>
#include <map>

using namespace std;

vector<double> probabilities = { 0.16, 0.09, 0.07, 0.05, 0.036, 0.036, 0.018, 0.018,
0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018,
0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018, 0.018,
0.018, 0.018, 0.018};
vector<char> symbols = {' ', 'а', 'л', 'и', 'р', 'ы', 'в', 'д', 'б', 'г', 'е', 'ж',
'з', 'й', 'к', 'м', 'н', 'о', 'п', 'с', 'т', 'у', 'ф', 'х', 'ц', 'ч', 'ш', 'щ', 'ь',
'э', 'ю', 'я', '!', ',', '?'};

string* Res = new string[35];

double schet1 = 0;
double schet2 = 0;

void sort() {
    for (int i = 0; i < 35; i++) {
        for (int j = 0; j < 35 - i - 1; j++) {
            if (probabilities[j] < probabilities[j + 1]) {
                char temp2;
                double temp1 = 0;

                temp1 = probabilities[j];
                temp2 = symbols[j];
                probabilities[j] = probabilities[j + 1];
                symbols[j] = symbols[j + 1];
                probabilities[j + 1] = temp1;
                symbols[j + 1] = temp2;
            }
        }
    }
}

int m;

int sequenceSplitting(int L, int R) {
    schet1 = 0;
    for (int i = L; i <= R - 1; i++) {
        schet1 = schet1 + probabilities[i];
    }

    schet2 = probabilities[R];
    m = R;
    while (schet1 >= schet2) {
        m = m - 1;
        schet1 = schet1 - probabilities[m];
        schet2 = schet2 + probabilities[m];
    }

    return m;
}

int g = 0;
```

```

void helpFano(int L, int R);

void methodShannonFano(int L, int R) {
    int n;

    if (L < R) {
        n = sequenceSplitting(L, R);
        for (int i = L; i <= R; i++) {
            if (i <= n) {
                Res[i] += '0';
            } else {
                Res[i] += '1';
            }
        }

        helpFano(L, n);

        methodShannonFano(n + 1, R);
    }
}

void helpFano(int L, int R) {
    int n;

    if (L < R) {
        n = sequenceSplitting(L, R);
        for (int i = L; i <= R; i++) {
            if (i <= n) {
                Res[i] += '0';
            } else {
                Res[i] += '1';
            }
        }

        methodShannonFano(L, n);

        helpFano(n + 1, R);
    }
}

map <char, int> numberEachSymbol(string text) {
    map <char, int> check;
    auto jt = check.begin();
    auto k = check.end();
    for (int i = 0; i < text.size(); i++) {
        bool flag = 0;
        for (auto it = check.begin(); it != k; it++) {
            if (it->first == text[i]) {
                it->second++;
                flag = 1;
            }
        }
        if (flag == 0) {
            check.insert(pair<char, int>(text[i], 1));
        }
    }

    return check;
}

float compressionRatio(map<char, int> check, int characters, int bitCharacter) {
    float sum = 0.0;
    int i = 0;
    for (auto it = check.begin(); it != check.end(); it++) {
        sum += Res[i].size() * it->second;
    }
}

```

```

        i++;
    }

    float result = (characters * bitCharacter) / (sum);

    return result;
}

float dispersion(map <char, int> check) {
    float Lsr = 0.0;
    int i = 0;

    for (auto it = 0; it < probabilities.size(); it++) {
        Lsr += Res[it].size() * probabilities[it];
    }

    float res = 0.0;
    for (auto it = 0; it < probabilities.size(); ++it) {
        res += probabilities[it] * ((Res[it].size() - Lsr) * (Res[it].size() -
Lsr));
    }

    return res;
}

int main() {
    setlocale(LC_ALL, "Russian");
    sort();
    methodShannonFano(0, 34);
    for (int i = 0; i < 35; i++) {
        cout << symbols[i] << " " << Res[i] << endl;
    }

    cout << symbols.size() << " " << probabilities.size();

    string text = "в чащах юга жил бы цитрус? Да но фальшивый экземпляр!";

    map <char, int> check = numberEachSymbol(text);

    float cR = compressionRatio(check, 8, symbols.size());

    float dis = dispersion(check);

    cout << "Коэффициент сжатия: " << cR << '\n';
    cout << "Дисперсия: " << dis << '\n';

    return 0;
}

```

Вывод программы:

```
Выбрать Консоль отладки Microsoft Visual Studio

000
а 001
л 01000
и 01001
р 0101
ы 01100
в 01101
д 01110
б 01111
г 1000000
е 1000001
ж 100001
з 10001
й 100100
к 100101
м 10011
н 101000
о 101001
п 10101
с 101100
т 101101
у 10111
ф 1100000
х 1100001
ц 110001
ч 11001
ш 110100
щ 110101
ь 11011
э 111000
ю 111001
я 11101
! 111100
, 111101
? 11111
35 35Коэффициент сжатия: 1.00719
Дисперсия: 1.58511
```

Вывод: коды Хаффмана и Шеннона-Фано – префиксные неравномерные коды. Часто встречающийся символ кодируется меньшим числом бит, редко – большим. Код Хаффмана и Шеннона-Фано в большинстве случаев дают одинаковый коэффициент сжатия, однако дисперсия кода Хаффмана стабильно ниже, чем у кода Шеннона-Фано.